

The Forge — inputs & outputs of the design (4 layers)

The trick to making game design un-fuzzy: separate *who* the input/output is for. The same "inputs → outputs" lives on four different layers, and they get conflated. Nail each one and the design stops being vague.

Tags: [✓] = modelled in the current sandbox · [→] = to design next.



Layer 1 — THE GAME (player ↔ system) — "what it feels like"

The actual Forge experience. Everything else exists to shape this.

INPUTS — what a player / faction can DO (the verbs):

- **Move** a squad toward a sector — costs travel-time over terrain [✓]
- **Capture** a sector — tap/hold to flip it (an NFC tap on the field) [✓]
- **Build a bridge** — spend resource to open a ravine crossing [✓]
- **Siege** — commit bodies to an enemy stronghold [✓]
- **Spend resource** on class abilities / intel / siege tools [→]
- **Scout / spend intel** — buy information, hide your own [→]
- **Parley** — strike (and break) deals between factions [→]
- **Pick a class / role** — what kind of unit you are [→]

INPUTS — what the world feeds the player (the state they read):

- Terrain: where you can go, how fast [✓]
- Current ownership, resources, momentum [✓]
- Time remaining in the 90-min session [✓]
- What you can and can't see (fog / intel) [→]

OUTPUTS — what the system gives back:

- **State changes:** ownership flips, resource ticks, bridges appear, strongholds fall [✓]
- **Feedback signals:** capture meters / LED arrays / the app / the GM's narration [✓ meters]
- **Score and win / lose / draw** [✓]
- **The experience** — tension, comeback, mastery, "I outplayed them." *This is the product.* The numbers above are only in service of this.

Design questions to chew on: Which verbs actually exist? Is capture a *tap* or a *sustained hold*? Is information a verb (scout) or passive? How many simultaneous decisions should one player juggle before it's soup?



Layer 2 — THE DESIGN (designer ↔ rules) — "the dials you own"

This is the job. Layer 1 is the **output** of the choices you make here.

INPUTS — what you, the designer, set:

- **Numeric levers:** capture rate · resource yield · bridge cost · travel speed · siege threshold · squads/faction · comeback aid · session length [✓ — **the 8 sliders**] · class stats · intel decay · parley payoffs [→]
- **Structural rules:** the central ravine · bridges-are-shared · no-eliminations ·

the scoring formula [✓]

- **The map / terrain** (real DEM) [→ **Stage 1**]

OUTPUTS — what you produce (this is SEED's literal deliverable):

- Rule structures + a living **design doc** + the **honesty ledger**

(intent / trade-off / failure mode per rule) [✓]

- A **balance verdict:** is it fair, deep, and alive?



Layer 3 — THE BALANCE MODEL (the measurement instrument = the sandbox)

A pure function: Layer-2 params in → balance metrics out.

INPUTS: design params · map (DEM) · agent policies · seed · # of matches **OUTPUTS — the metrics that decide the verdict:**

- Faction fairness % (is the map biased?)
- Dominant-strategy spread (is one opening strictly best?)

- Snowball % vs comeback % (is it decided early?)
- Dead-time % (how much of the session is nothing happening?)
- Siege-fire rate % (does the endgame trigger?)
- Lead-changes · matchup matrix



Layer 4 — THE RL AGENT (the smart playtester)

The new layer. Its I/O **falls straight out of Layer 1**: a *player's* inputs and outputs ARE an *agent's* observation and action. So designing the player nails the RL spec for free.

- **INPUT = observation** (what it sees each step): own + enemy ownership vector ·

resources · squad positions · bridge states · time left · terrain features · (later) visible intel.

- **OUTPUT = action:** a high-level command per squad (which sector to target) +

a build-bridge decision. Discrete and small → learns fast at low scale.

- **REWARD:** final score margin (+ light shaping for territory / momentum).

Self-play → it discovers exploits your heuristics missed.



The one line that ties it together

Layer 1's "what can a player do and see" literally defines Layer 4's observation/action space, and is the

thing Layer 2 tunes and Layer 3 measures. That's why thinking about the *game* first (Layer 1) is the right move — every other layer is downstream of it.